

Sucuri

Detecção de anomalias em despesas federais com Ensino Superior
Relatório consolidado de análises, medidas e interpretações

Portal da Transparência (gov.br) — Ministério da Educação

Atualizado em 16 de julho de 2026

Resumo

Este documento é a versão em $\text{\LaTeX}$ de `relatorios/RELATORIO.md` e deve conter, ao final do projeto, todas as análises, medidas estatísticas e interpretações produzidas ao longo das fases do `ROADMAP.md`. A versão atual documenta a **Fase 0 — Infraestrutura do projeto** (Parte I, que prepara a base de código, sem conclusões sobre despesas) e a **Fase 1 — Qualidade e preparação dos dados** (Parte II, primeira parte com interpretação de dados reais de despesas: qualidade dos dados, deflacionamento pelo IPCA e tratamento de ano parcial/séries curtas).

Sumário

1	Objetivo desta fase	2
2	Tarefa 0.1 — Empacotamento e ambiente	2
2.1	Estrutura criada	2
2.2	Migração de código	2
2.3	Dependências	2
2.4	Verificação dos critérios de aceite	2
3	Tarefa 0.2 — Testes unitários das funções críticas	2
3.1	Cobertura	2
3.2	Resultado da execução	4
4	Validação de não regressão	4
5	Ressalvas e decisões registradas	4
6	Estado do ROADMAP	4
	Parte II — Fase 1: Qualidade e preparação dos dados	5
7	Tarefa 1.1 — Relatório de qualidade dos dados	5
8	Tarefa 1.2 — Deflacionamento pelo IPCA	6
9	Tarefa 1.3 — Tratamento do ano parcial e revisão das flags	6
10	Estado do ROADMAP após a Fase 1	7
	Instrução de manutenção deste relatório	7

1 Objetivo desta fase

A Fase 0 do ROADMAP não produz análises sobre as despesas em si — seu objetivo é transformar o projeto de um único script (`coletar_despesas.py`) em uma base de código reprodutível, testável e reutilizável pelas fases seguintes (qualidade de dados, deflação, detecção de anomalias). Esta seção documenta **o que foi feito**; interpretações sobre os dados de despesas começam na Fase 1/2 e serão anexadas como novas seções deste mesmo documento.

2 Tarefa 0.1 — Empacotamento e ambiente

2.1 Estrutura criada

```
pyproject.toml # projeto gerenciado por uv (PEP 621 + hatchling)
uv.lock # lockfile de dependencias
src/sucuri/
  __init__.py
  api.py # cliente da API do Portal da Transparencia
  features.py # engenharia de variaveis e construação dos painéis A/B
  persistencia.py # salvamento de CSV/Parquet/JSON bruto e dicionario
  utils.py # brl_para_float, razao_segura, classificar_instituicao
analises/ # reservado para as fases 1-2 (vazio nesta fase)
relatorios/
  figuras/ # reservado para as fases 1-2 (vazio nesta fase)
  latex/ # este relatorio em LaTeX/PDF
tests/
  __init__.py
  test_utils.py
  test_features.py
coletar_despesas.py # CLI fino: apenas argparse + orquestracao
```

2.2 Migração de código

Toda a lógica reutilizável de `coletar_despesas.py` foi movida para o pacote `src/sucuri/`, preservando exatamente o comportamento original (mesmos parâmetros de API, mesmas fórmulas, mesmo formato de saída). A Tabela 1 resume o mapeamento.

O arquivo `coletar_despesas.py` (antes `coletar_despesas_ensino_superior.py`) ficou reduzido a aproximadamente 125 linhas: apenas `argparse`, orquestração das coletas (A) e (B), e chamadas às funções do pacote. Nenhuma fórmula ou regra de negócio foi alterada durante a migração — apenas reorganizada em módulos.

2.3 Dependências

Gerenciadas por uv: `pandas`, `pyarrow`, `requests`, `python-dotenv`, `scikit-learn`, `scipy`, `matplotlib` (produção); `ruff`, `pytest` (grupo dev). O comando `uv sync -group dev` cria o ambiente virtual e resolve o lockfile em poucos segundos.

2.4 Verificação dos critérios de aceite

3 Tarefa 0.2 — Testes unitários das funções críticas

3.1 Cobertura

Foram escritos 37 testes em dois arquivos, cobrindo exatamente as funções indicadas no ROADMAP.

Função/constante	Origem (script único)	Destino
<code>brl_para_float</code> , <code>_razao_segura</code> , <code>classificar_instituicao</code>	topo do script	<code>sucuri/utils.py</code> (<code>razao_segura</code> pública) tornada
<code>carregar_chave_api</code> , <code>criar_sessao</code> , <code>requisitar</code> , <code>coletar_paginado</code> , <code>coletar_intervalo</code>	seção “Coleta genérica”	<code>sucuri/api.py</code>
<code>construir_df_funcional</code> , <code>construir_df_instituicao</code> , <code>_indicadores_execucao</code> , <code>_features_serie_temporal</code> , <code>_consolidar_flags</code>	seções (A)/(B) e “Engenharia de variáveis”	<code>sucuri/features.py</code> (funções tornadas públicas)
<code>salvar_dataset</code> , <code>escrever_dicionario</code> , <code>DICIONARIO</code> , <code>resumir</code>	seção “Persistência”	<code>sucuri/persistencia.py</code>

Tabela 1: Mapeamento das funções migradas do script único para o pacote `sucuri`.

Critério (ROADMAP 0.1)	Resultado
<code>uv run python coletar_despesas.py -help</code> funciona	OK
<code>uv run pytest</code> roda (mesmo com 0 testes)	OK
<code>uv run ruff check .</code> sem erros	OK

tests/test_utils.py (23 testes)

- `brl_para_float`: formato brasileiro com separador de milhar (1.059.473.395,24), sem separador de milhar, `None`, string vazia, string só com espaços, valor já numérico (`int/float`), entrada inválida (`"abc"` → `NaN`), valor negativo.
- `razao_segura`: divisão normal e **denominador zero** → `NaN` (em vez de erro ou `inf`).
- `classificar_instituicao`: um caso por categoria (Universidade Federal, Hospitalar/EB-SERH, Instituto Federal, CEFET, CAPES, Fundo FNDE/FIES, Educação Básica, Outros/Administração), mais `None` e string vazia.

tests/test_features.py (14 testes)

- `indicadores_execucao`: taxas e saldos em caso normal; `empenhado == 0` → **taxas NaN sem exceção**; disparo isolado de cada flag de incoerência (`pago > empenhado`, `liquidado > empenhado`, valor negativo); ausência de falso positivo em valores coerentes.
- `features_serie_temporal`: variação anual ano a ano; disparo de `flag_salto_anual` acima de 100%; **série com desvio-padrão zero (valores constantes) não gera erro de divisão por zero** e produz `zscore` `NaN` em vez de indefinido; série com uma única observação; independência entre séries diferentes agrupadas no mesmo `DataFrame`.
- `consolidar_flags`: nenhuma flag ativa, uma flag ativa, flag extra (`flag_atipico_entre_pares`) considerada, valores nulos tratados como `False` na consolidação.

Os dois casos de borda explicitamente pedidos no ROADMAP — **divisão por zero** e **série com desvio-padrão zero** — estão cobertos e passam sem exceções, confirmando que as fórmulas

originais (`.replace(0, pd.NA)`) já protegiam contra isso; os testes tornam essa garantia explícita e verificável automaticamente.

3.2 Resultado da execução

```
$ uv run pytest -v
===== 37 passed in 0.67s =====

$ uv run ruff check .
All checks passed!
```

Aceite (ROADMAP 0.2): **OK** `uv run pytest` verde; as funções citadas no ROADMAP (`brl_para_float`, `classificar_instituicao`, `_indicadores_execucao`, `_features_serie_temporal`, `_consolidar_flags`) têm cobertura direta.

4 Validação de não regressão

Após a migração, foi confirmado por busca no repositório que não restou nenhuma referência às funções privadas antigas (`_indicadores_execucao`, `_features_serie_temporal`, `_consolidar_flags`, `_razao_segura`) fora do pacote `sucuri`, e que os módulos migrados importam e executam corretamente. Os dados já coletados em `dados/` (dois conjuntos, 2014–2026) não foram alterados nesta fase — nenhuma coleta foi executada.

5 Ressalvas e decisões registradas

- As funções antes privadas (`_indicadores_execucao` etc.) foram tornadas públicas (sem `_`) ao mover para `sucuri/features.py`, pois passam a ser a API pública do pacote reutilizada pelas fases seguintes. Isso é uma mudança de visibilidade, não de comportamento.
- `analises/` e `relatorios/figuras/` foram criados vazios nesta fase — serão populados a partir da Fase 1/2 do ROADMAP.
- Nenhum dado foi recoletado; a chave `GOVBR_API_KEY` não foi lida nem impressa durante esta fase (apenas os testes unitários, que não fazem chamadas de rede, foram executados).

6 Estado do ROADMAP

Tarefas 0.1 e 0.2 marcadas como concluídas em `ROADMAP.md`.

Parte II — Fase 1: Qualidade e preparação dos dados

Data: 16 de julho de 2026

Responsável pela execução: modelo de linguagem (Sonnet 5), seguindo `CLAUDE.md` e `ROADMAP.md`

Esta parte cobre as tarefas 1.1, 1.2 e 1.3 do ROADMAP — a primeira parte do relatório com interpretação de dados reais de despesas. Ao final desta fase, o conjunto recomendado para toda análise de anomalia (Fase 2 em diante) passa a ser `dados/*_v2.{csv,parquet}`, não mais os arquivos originais da Fase 0.

7 Tarefa 1.1 — Relatório de qualidade dos dados

Método: `analises/01_qualidade.py` roda sobre os dados como saíram da Fase 0 (nominais, sem tratamento) e produz `relatorios/01_qualidade.md` com seis checagens: cobertura por ano, % de zeros por coluna monetária, séries curtas, duplicatas, incoerências de execução (`pago > empenhado` etc.) e uma checagem descritiva de ordem de grandeza.

Resultados e interpretação:

1. **Cobertura por ano** — confirma que **2026 é ano parcial**: no Conjunto B (todos os órgãos do MEC), o total pago cai de R\$ 204,7 bi em 2025 para R\$ 114,6 bi em 2026 (coleta feita em julho/2026) — não é uma queda de gasto, é cobertura incompleta do exercício. Já era uma ressalva conhecida em `CLAUDE.md`; a tarefa 1.1 apenas quantifica o efeito.
2. **Zeros** — Conjunto A tem ~79–81% de zeros nas colunas monetárias (granularidade fina: programa × ação, muitas combinações sem execução em um dado ano); Conjunto B tem 0% de zeros (granularidade grossa: total do órgão). Não é erro, é a natureza dos dois endpoints.
3. **Séries curtas** — 125 de 239 séries do Conjunto A (52,3%) têm menos de 5 anos de observação. Média/desvio-padrão calculados sobre 1–4 pontos não sustentam um z-score confiável — tratado na tarefa 1.3 (`serie_curta`).
4. **Duplicatas** — **achado novo desta tarefa, não estava documentado antes**. O Conjunto A tem 69 linhas duplicadas em (`ano`, `chave_serie`). Investigação da causa-raiz: não são séries diferentes colidindo por acaso — é a API do Portal da Transparência retornando, para o mesmo `codigoPrograma/codigoAcao`, duas grafias do nome do programa/ação (ex.: “BRASIL UNIVERSITARIO” vs. “BRASIL UNIVERSITÁRIO”, “BOLSA PERMANENCIA” vs. “BOLSAPERMANENCIA” sem espaço) — aparentemente uma correção de texto na fonte que não substituiu o registro antigo. Em 67 dos 69 grupos os dois registros têm valores idênticos (em geral ambos zero); em **2 grupos os valores divergem** — um registro carrega o valor real e o outro fica zerado (exemplo: “CONCESSÃO DE BOLSA PERMANÊNCIA”, 2014, R\$ 77,5 milhões empenhados na grafia correta vs. R\$ 0 na grafia truncada). **Risco concreto:** se não tratado, um detector de anomalia ingênuo poderia confundir a alternância entre as duas grafias com um “salto anual” de 0 para um valor alto — um falso positivo criado por duplicidade de texto na fonte, não por um evento de gasto real. Corrigido na tarefa 1.3 por agregação (soma) das duplicatas antes de qualquer cálculo de série temporal.
5. **Incoerências de execução** — apenas 1 linha em todo o Conjunto A com `pago > empenhado` (2026, ação de regulação/supervisão de cursos: empenhado R\$ 67, pago R\$ 103.240) e nenhuma no Conjunto B. Amostra pequena demais para conclusão — fica como candidata a checagem manual; nenhuma linha com valor negativo em nenhum dos conjuntos.

6. **Ordem de grandeza** — total pago em 2025: ~R\$ 204,7 bi (Conjunto B, MEC completo) e ~R\$ 41,1 bi (Conjunto A, só subfunção 364). Ambos compatíveis com a ordem de grandeza publicamente conhecida do orçamento do MEC (dezenas de bilhões de reais/ano) — sem indício de erro de unidade. Validação linha a linha contra a dotação do SIOP fica para a tarefa 4.2.

Relatório completo: `relatorios/01_qualidade.md`.

8 Tarefa 1.2 — Deflacionamento pelo IPCA

Método: o pré-requisito (IPCA anual) foi **automatizado** em `analises/00_baixar_ipca.py` — baixa a série mensal do IPCA (Banco Central, SGS série 433, API pública sem cadastro) e calcula o acumulado por ano civil. `src/sucuri/deflacao.py` constrói um índice de preços encadeado a partir desses acumulados anuais e converte valores nominais em reais do ano-base (razão entre os índices de dois anos quaisquer = fator de correção inflacionária entre eles). `analises/01b_deflacionar.py` aplica isso aos dois conjuntos, gerando `empenhado_real/liquidado_real/pago`

Resultados e interpretação:

- **IPCA acumulado por ano** (2014–2026, sendo 2026 parcial — só janeiro a junho): variação anual entre 2,9% (2017) e 10,7% (2015), com dois anos de inflação alta (2015: 10,7%; 2021: 10,1%) intercalados por anos mais estáveis (2017–2019: 3,0–4,3%). Acumulado 2014→2025: fator de correção de **1,824×** — ou seja, R\$ 1,00 de 2014 equivale a R\$ 1,82 de poder de compra de 2025.
- **Ano-base escolhido: 2025** (último ano com os 12 meses de IPCA disponíveis — 2026 é parcial tanto nos dados de despesa quanto no IPCA, então não pode ser base de comparação).
- **Checagem de sanidade (critério de aceite do ROADMAP):** valores de 2014 ficam maiores após a correção — pago nominal de 2014 no Conjunto A é R\$ 25,89 bi; pago real (R\$ de 2025) é **R\$ 47,21 bi** (fator 1,824, batendo com o fator do IPCA acumulado). Mesmo fator no Conjunto B (R\$ 104,71 bi nominal → R\$ 190,94 bi real). Confirma que a implementação está correta: a inflação de 12 anos quase dobra o valor nominal antigo quando expresso em reais de hoje.
- **Interpretação para a Fase 2:** qualquer comparação de crescimento de gasto entre anos distantes (ex.: 2014 vs. 2025) DEVE usar as colunas `*_real`; usar as colunas nominais superestimaria o crescimento real em até ~82 pontos percentuais só pelo efeito da inflação acumulada.

9 Tarefa 1.3 — Tratamento do ano parcial e revisão das flags

Método: `analises/01c_ano_parcial_e_flags.py` parte de `dados/*_real.*` e (1) deduplica o Conjunto A somando as colunas monetárias das linhas identificadas na tarefa 1.1 (mesma `chave_serie/ano`, grafias diferentes); (2) marca `ano_parcial=True` no ano detectado pelo carimbo de `dados/raw/*.json` (`sucuri.persistencia.detectar_ano_coleta`); (3) marca `serie_curta=True` nas séries com menos de 5 anos de observação; (4) recalcula `variacao_pago_aa`, `zscore_pago`, `zscore_robusto_pago` e (só Conjunto B) `zscore_pago_entre_pares` — **com base em pago_real, não mais pago nominal** — usando como base estatística só as linhas elegíveis (nem `ano_parcial` nem `serie_curta`); as demais linhas recebem NaN nessas métricas e `False` nas flags derivadas.

Decisão de desenho não prevista literalmente no ROADMAP, registrada aqui: a tarefa 1.2 já deflaciona os valores; fazia sentido que a tarefa 1.3, ao recalculer z-scores/variação anual,

usasse `pago_real` em vez de `pago` nominal — comparar valores nominais entre anos distantes misturaria inflação com variação real de gasto, o problema que a tarefa 1.2 existe para resolver. As funções de `src/sucuri/features.py` foram generalizadas (parâmetro `coluna_valor`) para suportar isso sem duplicar código nem quebrar o comportamento (testado) da Fase 0.

Resultados e interpretação:

Conjunto	Linhas antes	Linhas depois	flag_anomalia antes ¹	flag_anom
A (funcional-programático)	1.318	1.249 (−69)	41	
B (por instituição)	1.484	1.484 (sem duplicatas)	73	

Tabela 2: Contagem de flags de anomalia antes e depois da tarefa 1.3. ¹ Pipeline original da Fase 0: nominal, sem dedup, sem exclusão de ano parcial/série curta. ² Pipeline da tarefa 1.3: `pago_real`, deduplicado, excluindo ano parcial (2026) e séries curtas da base estatística.

- **A queda de 41→19 no Conjunto A** vem de três efeitos somados: a deduplicação (remove falsos “saltos” de grafia), a exclusão do ano parcial (remove quedas artificiais de 2026) e a exclusão de séries curtas (remove z-scores calculados sobre 1–4 pontos, estatisticamente instáveis). Não é possível, com os dados agregados no log, atribuir a queda a um único efeito isoladamente — isso fica registrado como limitação e pode ser decomposto em uma iteração futura se for necessário isolar a contribuição de cada fator.
- **A queda de 73→58 no Conjunto B** vem só da exclusão do ano parcial (o Conjunto B não tem duplicatas nem séries curtas — todas as ~115 instituições têm histórico desde 2014).
- **Nenhuma flag de anomalia remanescente é baseada no ano parcial** — critério de aceite do ROADMAP cumprido: toda linha com `ano_parcial=True` tem `flag_anomalia_zscore=False`, `flag_anomalia_robusto=False`, `flag_salto_anual=False` e (no Conjunto B) `flag_atipico_entre_pare` por construção (excluídas do cálculo, não apenas filtradas depois).
- `dados/DICIONARIO.md` foi atualizado com uma nova seção descrevendo os conjuntos `_real` e `_v2`, deixando explícito que `_v2` é o conjunto recomendado para a Fase 2 — os demais (`_real` e os originais da Fase 0) são estágios intermediários do pipeline, não o ponto de partida para detecção de anomalias.

10 Estado do ROADMAP após a Fase 1

Tarefas 1.1, 1.2 e 1.3 marcadas como concluídas em `ROADMAP.md`, com os desvios de escopo (deduplicação do Conjunto A, uso de `pago_real` nas métricas de série temporal) documentados tanto lá quanto nesta seção. Próxima tarefa pendente: **2.1 Análise exploratória com foco em anomalias** (`analises/02_eda.py` → `relatorios/02_eda.md`), que deve partir de `dados/*_v2.{csv,parquet}`.

Instrução de manutenção deste relatório

A partir da Fase 1, toda nova tarefa do ROADMAP que gerar análise, medida estatística ou modelo deve acrescentar uma seção a este documento (e à versão Markdown em `relatorios/RELATORIO.md`) com: o que foi medido, o método usado, o resultado numérico e sua interpretação — nunca apenas o código ou o caminho do arquivo gerado. Ver `CLAUDE.md`, seção “Registro de análises e relatórios”.